
APACHE AIRFLOW 3.1.7



STAGE 2 / ARCHITECTURE

Architecture Deep Dive

Scheduler, Executor, Metadata Database & XCom

Target:	Backend / Distributed Systems Engineer
Duration:	Ngày 3-5 (2-3 giờ/ngày)
Focus:	Internal mechanics
Version:	Apache Airflow 3.1.7

Mục lục

1. Tổng quan kiến trúc

2. Scheduler Loop

2.1. 7 bước trong scheduler loop

2.2. Cấu hình tối ưu scheduler

3. Metadata Database Schema

3.1. Các bảng quan trọng

3.2. XCom và DB bloat

4. Executor Architecture

4.1. So sánh các Executor

4.2. Khi nào dùng Executor nào

5. XCom Deep Dive

5.1. Push/pull pattern

5.2. Custom XCom Backend với Redis

6. Sensors & Deferrable Pattern

6.1. 3 chế độ sensor

6.2. Triggerer service

7. Bài tập thực hành

1. Tổng quan kiến trúc

Để vận hành Airflow hiệu quả trong production, bạn cần hiểu rõ các thành phần cốt lõi và cách chúng tương tác. Stage 2 đi sâu vào 4 trụ cột của Airflow architecture:

- **Scheduler** — Bộ não, quyết định task nào chạy và khi nào
- **Executor** — Cánh tay, thực thi task trên worker
- **Metadata Database** — Bộ nhớ, lưu trạng thái toàn bộ hệ thống
- **XCom** — Hệ thống truyền thông điệp giữa các task

Airflow 3.x có kiến trúc **service-oriented** với Task Execution API (AIP-72) tách biệt việc thực thi task khỏi scheduler. Đây là thay đổi kiến trúc quan trọng nhất kể từ Airflow 2.0.

2. Scheduler Loop

Scheduler là thành phần quan trọng nhất trong Airflow. Nó chạy một vòng lặp vô hạn, mỗi vòng lặp gọi là một **scheduler loop**, thực hiện 7 bước để quyết định task nào sẽ chạy.

2.1. 7 bước trong scheduler loop

Bảng 1: 7 bước trong scheduler loop

Bước	Mô tả	Chi tiết
1	Parse DAG files	Đọc và parse các file Python trong <code>dags_folder</code>
2	Tạo DagRun	Kiểm tra schedule — nếu đến giờ, tạo DagRun mới
3	Tạo TaskInstance	Tạo 1 TaskInstance cho mỗi task trong DagRun
4	Kiểm tra dependencies	Upstream success, sensors, trigger rules
5	Queue task	Đẩy ready TaskInstance vào Executor queue
6	Cập nhật trạng thái	Theo dõi state từ Executor, cập nhật DB
7	Kiểm tra hoàn thành	DagRun hoàn thành / thất bại / đang chạy

Mỗi vòng lặp diễn ra liên tục, với độ trễ được điều khiển bởi `SCHEDULER_HEARTBEAT_SEC` (mặc định 5 giây). Scheduler heartbeat là cơ chế để các component khác biết scheduler vẫn

đang hoạt động.

Scheduler là điểm đơn nghẽn (single point of bottleneck)

Nếu scheduler quá tải, toàn bộ pipeline sẽ chậm lại. Hãy monitor `airflow_scheduler_heartbeat` và `dag_processing_total_parse_time`.

2.2. Cấu hình tối ưu scheduler

Các tham số sau ảnh hưởng trực tiếp đến hiệu năng scheduler:

```
# airflow.cfg hoặc environment variables
AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL = 120
AIRFLOW__SCHEDULER__MIN_FILE_PROCESS_INTERVAL = 30
AIRFLOW__SCHEDULER__MAX_DAGRUNS_PER_LOOP_TO_SCHEDULE = 20
AIRFLOW__SCHEDULER__PARSING_PROCESSES = 2
AIRFLOW__SCHEDULER__SCHEDULER_HEARTBEAT_SEC = 5
```

Bảng 2: Giải thích tham số scheduler

Tham số	Tác dụng	Khuyến nghị
<code>DAG_DIR_LIST_INTERVAL</code>	Tần suất kiểm tra file DAG mới/thay đổi	120s nếu DAG ít thay đổi
<code>MIN_FILE_PROCESS_INTERVAL</code>	Thời gian tối thiểu giữa 2 lần parse cùng file	30s để giảm CPU
<code>MAX_DAGRUNS_PER_LOOP</code>	Số DagRun tối đa xử lý mỗi vòng lặp	20 cho production
<code>PARSING_PROCESSES</code>	Số process song song để parse DAG	CPU cores / 2
<code>SCHEDULER_HEARTBEAT_SEC</code>	Tần suất heartbeat	5s (không nên < 3s)

3. Metadata Database Schema

Metadata database là trái tim của Airflow. Mọi trạng thái, mọi quyết định đều được ghi xuống DB. Hiểu các bảng quan trọng giúp bạn debug và tối ưu hiệu năng.

3.1. Các bảng quan trọng

Bảng 3: Các bảng quan trọng trong metadata database

Bảng	Mục đích	Query thường gặp
<code>dag</code>	Metadata DAG: <code>dag_id</code> , <code>fileloc</code> , <code>is_active</code> , <code>is_paused</code>	<code>SELECT dag_id FROM dag WHERE is_active=true</code>
<code>dag_run</code>	Mỗi lần chạy của DAG	<code>SELECT * FROM dag_run WHERE dag_id='x' ORDER BY execution_date DESC</code>
<code>task_instances</code>	State của từng task trong từng run	<code>SELECT state, COUNT(*) FROM task_instance GROUP BY state</code>
<code>xcom</code>	Cross-communication data giữa task	<code>SELECT * FROM xcom WHERE dag_id='x' AND execution_date=' ... '</code>
<code>job</code>	Scheduler/Worker heartbeat và trạng thái	Monitoring liveness
<code>connections</code>	Credentials mã hóa: <code>conn_id</code> , <code>conn_type</code> , <code>extra</code>	<code>SELECT conn_id FROM connection</code>
<code>variables</code>	Key-value config toàn cục	<code>SELECT val FROM variable WHERE key='env'</code>

3.2. XCom và DB bloat

Cảnh báo: Bảng `xcom` dễ bloat

Bảng `xcom` dễ phình to nếu task truyền dữ liệu lớn qua XCom. Mỗi giá trị XCom được lưu trực tiếp vào DB dưới dạng pickled bytes. Với data > 1KB, nên dùng **custom XCom backend** (Redis).

Giải pháp: Chỉ truyền *metadata* (URI, ID) qua XCom, không truyền *data*. Dữ liệu thực tế lưu trên Redis hoặc object storage.

4. Executor Architecture

Executor là thành phần quyết định *ở đâu* và *như thế nào* task được thực thi. Scheduler chỉ quyết định task *sẵn sàng* chạy; Executor quyết định task chạy *trên đâu*.

4.1. So sánh các Executor

Bảng 4: So sánh Executor trong Airflow 3.1.7

Executor	Mô hình process	Scaling	Phù hợp cho	Chi phí
SequentialExecutor	1 task tại 1 thời điểm (SQLite)	Không	Test, dev đơn giản	Không có
LocalExecutor	<code>fork()</code> trên 1 node	Vertical (CPU/RAM)	Dev, small prod (<50 DAGs)	Thấp
CeleryExecutor	Worker phân tán	Horizontal (Redis/Kafka)	Production, có Redis/Kafka	Trung bình
KubernetesExecutor	1 task = 1 Pod	K8s HPA	Isolation, dynamic workloads	Cao (K8s overhead)
EdgeExecutor (3.0+)	Remote/edge nodes	Event-driven	Hybrid cloud, IoT	Thay đổi

4.2. Khi nào dùng Executor nào

Bảng 5: Lựa chọn Executor theo use case

Tình huống	Executor phù hợp	Lý do
Dev trên laptop, <10 DAGs	LocalExecutor	Đơn giản, không cần infrastructure thêm
Production, team có Redis sẵn	CeleryExecutor	Horizontal scaling, có sẵn monitoring (Flower)
Multi-tenant, cần isolation	KubernetesExecutor	Mỗi task chạy trong pod riêng biệt
Task chạy ở edge/remote site	EdgeExecutor	Không cần full Airflow ở remote
Test nhanh, đơn giản	SequentialExecutor	Không cần Postgres, chỉ cần SQLite

5. XCom Deep Dive

XCom (Cross-communication) là cơ chế để các task trao đổi dữ liệu. Mặc định, XCom lưu dữ liệu vào metadata database, nhưng có thể cấu hình để dùng backend khác như Redis.

5.1. Push/pull pattern

Cú pháp cơ bản để truyền metadata giữa các task:

```

from airflow.operators.python import PythonOperator

def push_metadata(ti, **kwargs):
    """Push metadata qua XCom – chỉ nên truyền nhỏ, không data lớn."""
    ti.xcom_push(key='batch_id', value='INV-20260601-001')
    ti.xcom_push(key='redis_key', value='processed:001')

def pull_metadata(ti, **kwargs):
    """Pull metadata từ task khác."""
    batch_id = ti.xcom_pull(task_ids='generate_batch', key='batch_id')
    redis_key = ti.xcom_pull(task_ids='generate_batch', key='redis_key')
    print(f"Processing {batch_id} from Redis key {redis_key}")

# TaskFlow API – tự động xcom qua return/param
@task
def task_a() → str:
    return "result_a" # Tự động xcom_push

@task
def task_b(value: str) → str: # Tự động xcom_pull từ task_a
    return f"received: {value}"

```

5.2. Custom XCom Backend với Redis

Để tránh DB bloat khi truyền data lớn, hãy cấu hình Redis làm XCom backend. Thay vì S3 (như một số hướng dẫn), ta dùng Redis vì đã có sẵn trong stack:

```

# Cài đặt: pip install apache-airflow-providers-redis

# airflow.cfg
[core]
xcom_backend = airflow.providers.redis.xcom_backends.RedisXComBackend

[redis]
host = redis
port = 6379
password =
db = 1 # Dùng DB 1 cho XCom, tránh conflict với Celery (DB 0)

```

Hoặc dùng environment variable:

```

AIRFLOW__CORE__XCOM_BACKEND=airflow.providers.redis.xcom_backends.RedisXComBackend
AIRFLOW__REDIS__HOST=redis
AIRFLOW__REDIS__DB=1

```


Bảng 6: Best practice cho XCom

Kịch bản	Cách xử lý
Truyền metadata (<1KB)	XCom mặc định (DB) — ổn định
Truyền data trung bình (1KB-10MB)	Redis XCom backend
Truyền data lớn (>10MB)	Lưu Redis trước, XCom chỉ truyền key
Truyền kết quả giữa task	TaskFlow API — tự động, type-safe

6. Sensors & Deferrable Pattern

Sensor là loại Operator đặc biệt — nó *chờ đợi* một điều kiện xảy ra trước khi tiếp tục. Ví dụ: chờ file xuất hiện, chờ DAG khác hoàn thành, chờ message trên Kafka.

6.1. 3 chế độ sensor

Airflow hỗ trợ 3 chế độ cho sensor, mỗi chế độ có ưu/nhược điểm khác nhau:

```

from airflow.sensors.filesystem import FileSensor
from airflow.sensors.external_task import ExternalTaskSensor
from datetime import timedelta

# ❌ BAD: Poke mode – chiếm worker slot liên tục
wait_file = FileSensor(
    task_id='wait_for_file',
    filepath='/data/invoices/{{ ds }}/done.flag',
    poke_interval=60,
    timeout=timedelta(hours=2),
    mode='poke', # Mặc định – worker bị chiếm suốt thời gian chờ
)

# ✅ GOOD: Reschedule mode – release slot giữa các lần poke
wait_file_reschedule = FileSensor(
    task_id='wait_for_file_reschedule',
    filepath='/data/invoices/{{ ds }}/done.flag',
    poke_interval=60,
    timeout=timedelta(hours=2),
    mode='reschedule', # Worker được giải phóng giữa các lần kiểm tra
)

# ✅ BEST: Deferrable (Async) – không cần worker, dùng triggerer
wait_file_defer = FileSensor(
    task_id='wait_for_file_defer',
    filepath='/data/invoices/{{ ds }}/done.flag',
    poke_interval=60,
    timeout=timedelta(hours=2),
    deferrable=True, # Async – cần chạy airflow triggerer
)

```

Bảng 7: So sánh 3 chế độ sensor

Chế độ	Cách hoạt động	Tốn tài nguyên	Phù hợp
poke	Chiếm worker slot, kiểm tra liên tục	Cao — 1 worker slot bị chiếm	Không khuyến khích
reschedule	Release slot giữa các lần kiểm tra	Trung bình — chỉ chiếm lúc kiểm tra	Good — dùng khi không có triggerer
deferrable	Async — không chiếm worker, dùng triggerer	Thấp nhất — không chiếm worker	Best — yêu cầu chạy triggerer

6.2. Triggerer service

Để dùng deferrable operators/sensors, cần chạy thêm service `triggerer`:

```
# Thêm vào docker-compose.yml
airflow-triggerer:
  <<: *airflow-common
  command: triggerer
```

Cách deferrable hoạt động

Khi task gặp deferrable operator, nó **tạm dừng** (defer) và ghi trạng thái vào DB. Triggerer đọc DB, theo dõi điều kiện ở background. Khi điều kiện thỏa mãn, triggerer đánh thức task tiếp tục chạy.

7. Bài tập thực hành

Bảng 8: Bài tập Stage 2

STT	Bài tập	Kỹ năng
1	Query bảng <code>dag_run</code> và <code>task_instance</code> trong Postgres	Hiểu metadata schema
2	Push/pull XCom giữa 2 tasks, verify trong UI Admin → XComs	Thành thạo XCom
3	Thử cả 3 sensor modes: poke, reschedule, deferrable	Hiểu sensor behavior
4	Chạy <code>airflow triggerer</code> và test deferrable sensor	Cấu hình triggerer
5	Đo <code>dag_processing.total_parse_time</code> qua UI metrics	Monitoring scheduler
6	Thử đổi Executor từ Local → Sequential → observe difference	Hiểu executor architecture

Debug hiệu quả

Khi gặp vấn đề, hãy kiểm tra theo thứ tự: (1) Scheduler logs, (2) `dag_run` state trong DB, (3) `task_instance` state, (4) XCom values (nếu có). Câu lệnh `airflow tasks test <dag_id> <task_id> <date>` giúp test task độc lập mà không cần chạy cả DAG.